

Lecture 8: LL(1) parsing

David Hovemeyer

September 22, 2025

601.428/628 Compilers and Interpreters



Today

- ▶ LL(1) parsing

Top-down parsing

- ▶ A *top-down* parsing algorithm attempts to derive an input string by starting with the start symbol, and applying productions (and consuming terminal symbols from the input string) until no nonterminals remain in the working string
- ▶ Using the lexer for lookahead helps the parser choose productions correctly
- ▶ Recursive descent is a top-down parsing technique
- ▶ LL(1) is a *generalized* top-down parsing technique
- ▶ Given a suitable context-free grammar, an LL(1) parser can be *generated*

Parser generators

- ▶ A *parser generator* takes a context-free grammar as input, and generates code for a parser
- ▶ The context-free grammar can be augmented with *semantic actions* to be carried out as productions are chosen
 - ▶ E.g., the semantic actions could build a parse tree or AST
- ▶ Advantage to using a parser generator: less work!
 - ▶ Also, greater confidence in correctness of parser
- ▶ Disadvantage to using a parser generator: less control (especially for generating meaningful error messages)

LL(1)

LL(1) is a simple table-driven top-down parsing algorithm

Requirements:

- ▶ Grammar has no left recursion
- ▶ For each nonterminal symbol A , FIRST^+ sets of all productions on A are disjoint (explanation soon)

FIRST sets

For a symbol K , $\text{FIRST}(K)$ is the set of all terminal symbols which could begin an expansion of K

- ▶ Also, $\text{FIRST}(K)$ contains ϵ if K can expand to the empty string

Example grammar: $A \rightarrow a$ (A is the start symbol)

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow b$

$B \rightarrow \epsilon$

$C \rightarrow c e$

$C \rightarrow d e$

$\text{FIRST}(A) = \{a, b, c, d, \epsilon\}$

Computing FIRST set

The FIRST set of any terminal symbol t is trivially $\{t\}$

For any nonterminal A , $\text{FIRST}(A)$ can be computed as follows:

For each production of the form $A \rightarrow s_1 s_2 s_3 \dots s_n$:

- If s_1 is a terminal symbol, s_1 is in $\text{FIRST}(A)$

- If s_1 is a nonterminal, then all symbols in $\text{FIRST}(s_1)$ are also in $\text{FIRST}(A)$

- If for some i in $1 \dots n$, all symbols $s_1 \dots s_{i-1}$ are nonterminals which can expand to ϵ , then all symbols in $\text{FIRST}(s_i)$ are in $\text{FIRST}(A)$

- If each $s_1 \dots s_n$ are nonterminals which can expand to ϵ , then ϵ is in $\text{FIRST}(A)$

Generalizing FIRST to string of symbols

If β is a string of symbols $s_1, s_2, \dots, s_n$

Then $\text{FIRST}(\beta)$ is the union of $\text{FIRST}(s_1), \text{FIRST}(s_2), \dots, \text{FIRST}(s_i)$ where for all j such that $1 \leq j < i$, $\epsilon \in \text{FIRST}(s_j)$

► Except: $\epsilon \in \text{FIRST}(\beta)$ if and only if $\epsilon \in \text{FIRST}(s_k)$ for all $1 \leq k \leq n$

FOLLOW sets

For a nonterminal K , $\text{FOLLOW}(K)$ is the set of terminal symbols which could follow an expansion of K

- Useful for knowing when it is appropriate to apply an epsilon production

Example grammar: $A \rightarrow a B c$ (A is the start symbol)

$A \rightarrow C$

$C \rightarrow d B G f$

$B \rightarrow g$

$G \rightarrow h$

$G \rightarrow \epsilon$

$\text{FOLLOW}(A)$ is $\{ \text{eof} \}$

$\text{FOLLOW}(B)$ is $\{ c, f, h \}$

Note: *eof* is a special “end of file” token

Computing FOLLOW set

If S is the start symbol, then eof is in $FOLLOW(S)$.

If a production $A \rightarrow \alpha B \beta$ exists, then all symbols in $FIRST(\beta)$ except ϵ are in $FOLLOW(B)$.

If either

- ▶ a production $A \rightarrow \alpha B$ exists, or
- ▶ a production $A \rightarrow \alpha B \beta$ exists, and $FIRST(\beta)$ contains ϵ

then all symbols in $FOLLOW(A)$ are in $FOLLOW(B)$

Example FIRST and FOLLOW sets

Let's see the construction of FIRST and FOLLOW sets in action:

Grammar (start symbol is E):

$$E \rightarrow T E'$$

$$E' \rightarrow + T E'$$

$$E' \rightarrow - T E'$$

$$E' \rightarrow \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T'$$

$$T' \rightarrow / F T'$$

$$T' \rightarrow \epsilon$$

$$F \rightarrow i$$

$$F \rightarrow n$$

(Note that we removed the A nonterminal and its productions, since they would cause the grammar to be unsuitable for LL(1) parsing)

Example FIRST and FOLLOW sets

Grammar (start symbol is E):

$$E \rightarrow T E'$$

$$E' \rightarrow + T E'$$

$$E' \rightarrow - T E'$$

$$E' \rightarrow \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T'$$

$$T' \rightarrow / F T'$$

$$T' \rightarrow \epsilon$$

$$F \rightarrow i$$

$$F \rightarrow n$$

Symbol	FIRST	FOLLOW
+		
-		
*		
/		
i		
n		
E		
E'		
T		
T'		
F		

Example FIRST and FOLLOW sets

Grammar (start symbol is E):

$$E \rightarrow T E'$$
$$E' \rightarrow + T E'$$
$$E' \rightarrow - T E'$$
$$E' \rightarrow \epsilon$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T'$$
$$T' \rightarrow / F T'$$
$$T' \rightarrow \epsilon$$
$$F \rightarrow i$$
$$F \rightarrow n$$

Symbol	FIRST	FOLLOW
+	{ + }	—
-	{ - }	—
*	{ * }	—
/	{ / }	—
i	{ i }	—
n	{ n }	—
E	{ i, n }	{ eof }
E'	{ +, -, ϵ }	{ eof }
T	{ i, n }	{ +, -, eof }
T'	{ *, /, ϵ }	{ +, -, eof }
F	{ i, n }	{ *, /, +, -, eof }

FIRST⁺ sets

For a production $A \rightarrow \beta$, FIRST⁺($A \rightarrow \beta$) is

- ▶ FIRST(β) if $\epsilon \notin \text{FIRST}(\beta)$
- ▶ FIRST(β) $\cup$ FOLLOW(A) otherwise

Production	FIRST ⁺ set
$E \rightarrow T E'$	
$E' \rightarrow + T E'$	
$E' \rightarrow - T E'$	
$E' \rightarrow \epsilon$	
$T \rightarrow F T'$	
$T' \rightarrow * F T'$	
$T' \rightarrow / F T'$	
$T' \rightarrow \epsilon$	
$F \rightarrow i$	
$F \rightarrow n$	

Symbol	FIRST	FOLLOW
+	{ + }	—
-	{ - }	—
*	{ * }	—
/	{ / }	—
i	{ i }	—
n	{ n }	—
E	{ i, n }	{ eof }
E'	{ +, -, ϵ }	{ eof }
T	{ i, n }	{ +, -, eof }
T'	{ *, /, ϵ }	{ +, -, eof }
F	{ i, n }	{ *, /, +, -, eof }

FIRST⁺ sets

For a production $A \rightarrow \beta$, FIRST⁺($A \rightarrow \beta$) is

- ▶ FIRST(β) if $\epsilon \notin \text{FIRST}(\beta)$
- ▶ FIRST(β) $\cup$ FOLLOW(A) otherwise

Production	FIRST ⁺ set
$E \rightarrow T E'$	{ i, n }
$E' \rightarrow + T E'$	{ + }
$E' \rightarrow - T E'$	{ - }
$E' \rightarrow \epsilon$	{ ϵ , eof }
$T \rightarrow F T'$	{ i, n }
$T' \rightarrow * F T'$	{ * }
$T' \rightarrow / F T'$	{ / }
$T' \rightarrow \epsilon$	{ ϵ , +, -, eof }
$F \rightarrow i$	{ i }
$F \rightarrow n$	{ n }

Symbol	FIRST	FOLLOW
+	{ + }	—
-	{ - }	—
*	{ * }	—
/	{ / }	—
i	{ i }	—
n	{ n }	—
E	{ i, n }	{ eof }
E'	{ +, -, ϵ }	{ eof }
T	{ i, n }	{ +, -, eof }
T'	{ *, /, ϵ }	{ +, -, eof }
F	{ i, n }	{ *, /, +, -, eof }

FIRST⁺ sets intuitively

The FIRST⁺ set for a production $A \rightarrow \beta$ tells us:

- ▶ If we want to expand an occurrence of A ,
- ▶ What lookahead tokens indicate that $A \rightarrow \beta$ is the right production to apply

LL(1) parse tables

An LL(1) parse table indicates, for each nonterminal in a grammar, what production to apply based on what the next input token is

- ▶ Rows: nonterminal symbols
- ▶ Columns: terminal symbols and *eof*
- ▶ Entries: contain either a production number, or “invalid”

Building LL(1) parse table:

Mark all entries as “invalid”

For each production numbered p of the form $A \rightarrow \beta$

For each terminal $t \in \text{FIRST}^+(A)$

Put p in row A , column t

If $\text{eof} \in \text{FIRST}^+(A)$

Put p in row A , column eof

LL(1) parse table example

	Production	FIRST ⁺ set
(1)	$E \rightarrow T E'$	$\{ i, n \}$
(2)	$E' \rightarrow + T E'$	$\{ + \}$
(3)	$E' \rightarrow - T E'$	$\{ - \}$
(4)	$E' \rightarrow \epsilon$	$\{ \epsilon, eof \}$
(5)	$T \rightarrow F T'$	$\{ i, n \}$
(6)	$T' \rightarrow * F T'$	$\{ * \}$
(7)	$T' \rightarrow / F T'$	$\{ / \}$
(8)	$T' \rightarrow \epsilon$	$\{ \epsilon, +, -, eof \}$
(9)	$F \rightarrow i$	$\{ i \}$
(10)	$F \rightarrow n$	$\{ n \}$

	$+$	$-$	$*$	$/$	i	n	eof
E							
E'							
T							
T'							
F							

LL(1) parse table example

	Production	FIRST ⁺ set
(1)	$E \rightarrow T E'$	$\{ i, n \}$
(2)	$E' \rightarrow + T E'$	$\{ + \}$
(3)	$E' \rightarrow - T E'$	$\{ - \}$
(4)	$E' \rightarrow \epsilon$	$\{ \epsilon, eof \}$
(5)	$T \rightarrow F T'$	$\{ i, n \}$
(6)	$T' \rightarrow * F T'$	$\{ * \}$
(7)	$T' \rightarrow / F T'$	$\{ / \}$
(8)	$T' \rightarrow \epsilon$	$\{ \epsilon, +, -, eof \}$
(9)	$F \rightarrow i$	$\{ i \}$
(10)	$F \rightarrow n$	$\{ n \}$

	+	-	*	/	i	n	eof
E	-	-	-	-	1	1	-
E'	2	3	-	-	-	-	4
T	-	-	-	-	5	5	-
T'	8	8	6	7	-	-	8
F	-	-	-	-	9	10	-

LL(1) parsing

- ▶ Data structures are:
 - ▶ Sequence of terminal symbols (tokens)
 - ▶ Stack of symbols
- ▶ Start by pushing *eof* followed by the (nonterminal) start symbol

LL(1) parsing (continued)

- ▶ Repeatedly:
 - ▶ Inspect top of stack
 - ▶ If a terminal symbol, consume same symbol from input string and pop it from the stack (error if next input string symbol doesn't match)
 - ▶ If *eof*, make sure we're at end of input, if so, done (otherwise error)
 - ▶ If a nonterminal symbol:
 - ▶ Based on next input symbol, choose production number from table (error if entry is "invalid")
 - ▶ Pop nonterminal from stack
 - ▶ For each symbol on right hand side of chosen production from right to left, push it onto the stack

Example parse

[illegible]

- (1) $E \rightarrow T E'$
- (2) $E' \rightarrow + T E'$
- (3) $E' \rightarrow - T E'$
- (4) $E' \rightarrow \epsilon$
- (5) $T \rightarrow F T'$
- (6) $T' \rightarrow * F T'$
- (7) $T' \rightarrow / F T'$
- (8) $T' \rightarrow \epsilon$
- (9) $F \rightarrow i$
- (10) $F \rightarrow n$

	+	-	*	/	i	n	<i>eof</i>
E	-	-	-	-	1	1	-
E'	2	3	-	-	-	-	4
T	-	-	-	-	5	5	-
T'	8	8	6	7	-	-	8
F	-	-	-	-	9	10	-

Example parse

Rule	Stack	Input
–	<i>eof</i> E	$\wedge$ n - i / n
1	<i>eof</i> E' T	$\wedge$ n - i / n
5	<i>eof</i> E' T' F	$\wedge$ n - i / n
10	<i>eof</i> E' T' n	$\wedge$ n - i / n
→	<i>eof</i> E' T'	n $\wedge$ - i / n
8	<i>eof</i> E'	n $\wedge$ - i / n
3	<i>eof</i> E' T' -	n $\wedge$ - i / n
→	<i>eof</i> E' T'	n - $\wedge$ i / n
5	<i>eof</i> E' T' F	n - $\wedge$ i / n
9	<i>eof</i> E' T' i	n - $\wedge$ i / n
→	<i>eof</i> E' T'	n - i $\wedge$ / n
7	<i>eof</i> E' T' F /	n - i $\wedge$ / n
→	<i>eof</i> E' T' F	n - i / $\wedge$ n
10	<i>eof</i> E' T' n	n - i / $\wedge$ n
→	<i>eof</i> E' T'	n - i / n $\wedge$
8	<i>eof</i> E'	n - i / n $\wedge$
4	<i>eof</i>	n - i / n $\wedge$

- (1) $E \rightarrow T E'$
- (2) $E' \rightarrow + T E'$
- (3) $E' \rightarrow - T E'$
- (4) $E' \rightarrow \epsilon$
- (5) $T \rightarrow F T'$
- (6) $T' \rightarrow * F T'$
- (7) $T' \rightarrow / F T'$
- (8) $T' \rightarrow \epsilon$
- (9) $F \rightarrow i$
- (10) $F \rightarrow n$

	+	-	*	/	i	n	<i>eof</i>
E	–	–	–	–	1	1	–
E'	2	3	–	–	–	–	4
T	–	–	–	–	5	5	–
T'	8	8	6	7	–	–	8
F	–	–	–	–	9	10	–

Thoughts about LL(1)

Is LL(1) a “good” parsing algorithm?

- ▶ Inability to handle grammars with left recursion is annoying
- ▶ 1 token of lookahead isn't sufficient for some grammars
 - ▶ For example, in our original grammar, we had productions $A \rightarrow i = A$ and $A \rightarrow E$
 - ▶ $FIRST^+$ sets of both productions will contain i and n , so they will conflict in the LL(1) parse table
 - ▶ A recursive descent parser could just look ahead by 2 tokens
 - ▶ LL(k) parsing is a generalization of LL(1) to allow greater lookahead